Iterative Improvement Algorithms for the Linear Ordering Problem

INFO-H-413 — Heuristic Optimization — Implementation Exercise 1

Université Libre de Bruxelles — 2026

Luis Brunard (577721)

Abstract

We compare twelve iterative improvement configurations and two Variable Neighbourhood Descent (VND) variants on 78 benchmark instances of the Linear Ordering Problem. The neighbourhood choice has the largest effect on solution quality: insert dominates exchange, and both leave transpose far behind. First-improvement finds better local optima than best-improvement, at the cost of longer runtimes. CW initialisation consistently beats a random start. VND-TIE slightly improves on the best single-neighbourhood result. All comparisons are supported by Wilcoxon signed-rank tests.

---

1. Introduction

The Linear Ordering Problem (LOP) is a ranking problem. Given an $n \times n$ weight matrix C, the goal is to find a permutation π of n elements that maximises the sum of weights above the diagonal:

$$f(\pi) = \sum_{i<j} c_{\pi(i),\pi(j)}$$

We want i to appear before j whenever $c_{ij} > c_{ji}$. The problem is NP-hard, so we don't have methods that solve the problem in an acceptable amount of time for bigger instances. We use heuristics instead.

This exercise is about local search: start from some solution, make small improving changes, and stop when none is possible. That stopping point is a **local optimum**. The question we are trying to answer is which combination of choices using neighbourhood, pivoting rule, initialisation, gets us to the best local optimum, and how quickly.

2. Problem Description

Three parameters control how local search behaves on the LOP.

2.1. Neighbourhood Structures

A neighbourhood defines which solutions are reachable in one step. We test three.

Transpose. Swap two adjacent elements. From a permutation of length n, there are $n-1$ such pairs. The gain Δ of swapping positions i and $i+1$ is:

$$\Delta_T(i) = c_{\pi(i+1),\pi(i)} - c_{\pi(i),\pi(i+1)}$$

Each evaluation is $O(1)$, making transpose by far the cheapest option.

Exchange. Swap any two elements, not necessarily adjacent. There are $\binom{n}{2}$ possible pairs, but computing each gain requires scanning everything between the two positions. $O(n)$ per move.

Insert. Remove an element from position i and reinsert it at position j, shifting what is in between. This gives $n(n-1)$ possible moves, also at $O(n)$ each.

2.2. Pivoting Rules

Once a neighbourhood is fixed, there are two ways to pick the next move.

- **First-improvement (FI):** apply the first improving move found and restart the scan immediately.
- **Best-improvement (BI):** scan the full neighbourhood, then apply the single best improving move.

2.3. Initialisation

- **Random:** a uniformly shuffled permutation.
- **Chenery–Watanabe (CW):** rank each element by its row-sum score $R_i = \sum_{j \neq i} c_{ij}$ and sort in decreasing order. The idea is to put elements that gain the most from being ranked early at the front, giving a starting point that is already better than a random shuffle.

3. Algorithms

3.1. Iterative Improvement (Exercise 1.1)

The algorithm is simple:

- Generate an initial solution π_0. ($\rightarrow$ Random/CW).
- While an improving neighbour exists:
  - (First-improvement) Apply the first move with $\Delta > 0$; restart scan.
  - (Best-improvement) Apply the move with the largest $\Delta > 0$.
- Return the local optimum π^*.

Combining 2 pivoting rules, 3 neighbourhoods, and 2 initialisations gives **12 configurations** in total.

3.2. Variable Neighbourhood Descent (Exercise 1.2)

The idea behind VND is that being stuck in a local optimum for one neighbourhood does not mean you are stuck for all of them. Instead of stopping, we switch to a different neighbourhood and keep searching. If an improvement is found, we go back to the first neighbourhood. We stop only when none of the neighbourhoods can improve the current solution.

We use first-improvement throughout, starting from the CW solution. Two orderings are tested:

- **VND-TEI:** Transpose $\rightarrow$ Exchange $\rightarrow$ Insert
- **VND-TIE:** Transpose $\rightarrow$ Insert $\rightarrow$ Exchange

4. Statistical Testing Method

4.1. Standard deviation

The **standard deviation** (SD) measures how spread a set of values is around their mean. A small SD means results are consistent across instances; a large one means performance depends heavily on the specific problem [1].

4.2. Wilcoxon test and p-value

Average RPDs (defined in section 5) alone can be misleading, a gap might come from a few unusual instances rather than a consistent trend. To check this, we use the **Wilcoxon signed-rank test** [2], a non-parametric test for paired data. For each instance we compute the RPD difference between two algorithms, rank those differences by absolute value, and check whether one algorithm wins consistently. The output is a **p-value**, the probability of seeing a gap this large if the two algorithms were actually equivalent. We use $\alpha = 0.05$: if $p < 0.05$, the difference is **statistically significant** [3]. All tests were run in R with the built-in `wilcox.test` function [4].

5. Results and Analysis

5.1. Experimental setup

The solver is written in C, compiled with GCC and `-O2` on macOS (Apple M1). Each of the 14 algorithm configurations was run once on all 78 instances. Compilation and execution details are in the `README.md` file included with the source code.

5.2. Evaluation metric

To compare solutions, we use the **Relative Percentage Deviation** (RPD) from the best-known solution:

$$\text{RPD} = \frac{\text{BestKnown} - \text{FinalCost}}{\text{BestKnown}} \times 100$$

An RPD of 0 means we matched the best-known value. A larger RPD means our solution is that many percent worse, so **lower is better** [5].

5.3. Exercise 1.1 — Iterative Improvement Results

Figure 1 shows solution quality for Insert, Exchange, and VND. Table 1 gives all numerical values for all 12 configurations.

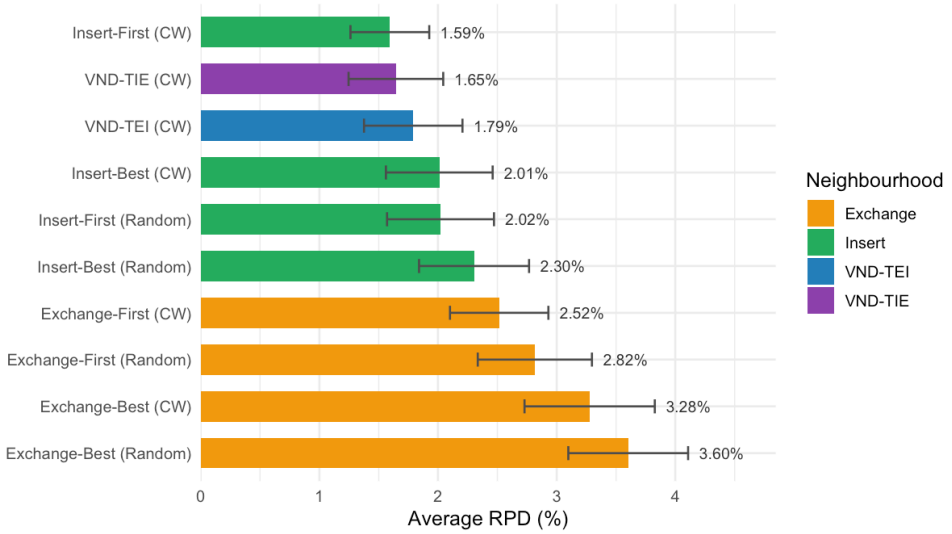

Figure 1: Average RPD (%) for Insert, Exchange and VND configurations, sorted from best to worst. Each bar shows the average over 78 instances. Transpose is excluded (RPD 19–35%) to keep the scale readable, it appears in Table 1 and Figure 7.

Algorithm	Init	Avg RPD	SD	Avg Time (s)	Total Time (s)
Insert — First	CW	1.59	0.33	2.99	233.4
Insert — Best	CW	2.01	0.45	0.79	61.7
Insert — First	Random	2.02	0.45	4.28	333.9
Insert — Best	Random	2.30	0.46	0.85	66.3
Exchange — First	CW	2.52	0.41	3.97	309.7
Exchange — First	Random	2.82	0.48	4.87	379.9
Exchange — Best	CW	3.28	0.55	0.70	54.3
Exchange — Best	Random	3.60	0.50	0.76	59.3
Transpose — Best	CW	19.25	1.89	0.006	0.5
Transpose — First	CW	19.38	1.92	0.003	0.2
Transpose — Best	Random	34.47	3.80	0.007	0.5
Transpose — First	Random	34.61	3.80	0.004	0.3

Table 1: All 12 configurations over 78 instances, sorted by solution quality.

5.3.1. Observations

The **neighbourhood** is clearly the dominant factor. Insert is the best choice, with RPDs between 1.6% and 2.3% depending on the pivoting rule and initialisation. Exchange comes in second at 2.5–3.6%, and transpose is far behind at 19–35%. Transpose can only swap adjacent elements, so it needs an enormous number of moves to rearrange a permutation significantly and it gets stuck long before reaching a decent solution.

Initialisation matters too, and the effect shows up in every configuration. CW consistently beats a random start, and the gap is largest for transpose (19% vs 34%). This makes sense: when

a neighbourhood can only shift elements one position at a time, the starting point determines almost everything.

The more surprising result is the pivoting rule. For insert, **first-improvement finds better solutions than best-improvement** (1.59% vs 2.01% with CW). The reason has to do with the shape of the search space. Best-improvement always jumps to the steepest available gain, but the steepest gain is not always toward the global optimum.

It can commit to a local optimum early, with no way back. First-improvement is more conservative: it takes the first move that works, which keeps it from over-committing and gives it more room to explore before settling.

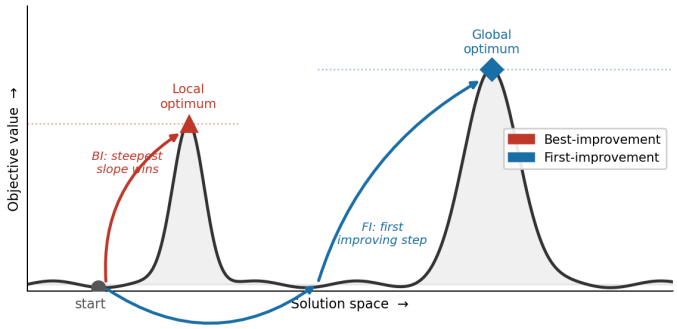

Figure 2: Difference between BI and FI

On **time**, the situation is reversed. Best-improvement (0.79 s for Insert-CW) is roughly 4× faster than first-improvement (2.99 s). This sounds strange given that BI inspects the full neighbourhood at every step, but the key is the number of iterations. Because BI always takes the biggest gain available, it climbs faster and reaches a local optimum in far fewer steps. FI makes many small moves and the cumulative cost adds up. The time column in Table 1 shows this pattern clearly: FI variants are the slowest, BI variants sit in the middle, and transpose barely registers.

5.3.2. Statistical Tests

Pairwise Wilcoxon signed-rank tests were applied at significance level $\alpha = 0.05$. A representative selection of results is shown in Table 2.

Algorithm A	**Algorithm B**	**p-value**	**Significant?**
Insert-First (CW)	Transpose-First (CW)	< 0.001	Yes
Insert-First (CW)	Exchange-First (CW)	< 0.001	Yes
Insert-First (CW)	Insert-Best (CW)	< 0.001	Yes
Insert-First (CW)	Insert-First (Random)	< 0.001	Yes
Insert-Best (CW)	Insert-First (Random)	0.9404	**No**
Exchange-First (CW)	Exchange-Best (CW)	< 0.001	Yes
Transpose-First (CW)	Transpose-First (Random)	< 0.001	Yes

Table 2: Selected pairwise Wilcoxon signed-rank test results ($\alpha = 0.05$). All 66 pairs were tested, nearly all are significant except the highlighted case.

Almost every pair is significantly different ($p < 0.001$), which is what we would expect given the large gaps in Table 1. The exception is **Insert-Best (CW) vs Insert-First (Random)** ($p = 0.94$): these two configurations use completely different strategies. One relies on a smart start and a weaker search, the other on a random start and a thorough one, yet they land at

essentially the same quality (2.01% vs 2.02%). A good starting point can compensate for a weaker algorithm, at least in this case.

Figure 3 shows this visually: CW gives tighter, lower RPD distributions than a random start, regardless of neighbourhood or pivoting rule.

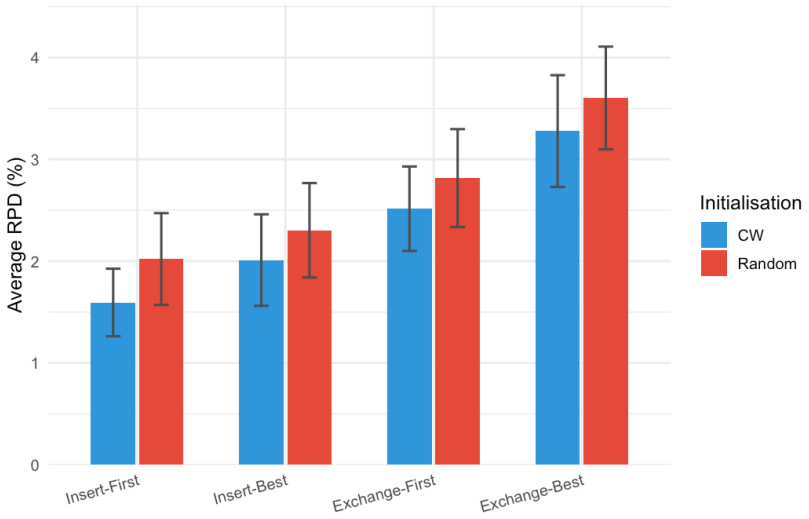

Figure 3: Average RPD (%) for CW initialisation and random initialisation, for each neighbourhood and pivoting rule combination.

5.4. Exercise 1.2 — VND Results

Table 3 and Figure 4 show the results for both VND orderings, starting from CW.

Algorithm	Init	Avg RPD	SD	Avg Time (s)	Total Time (s)
VND-TIE	CW	1.65	0.40	3.24	252.4
VND-TEI	CW	1.79	0.42	4.86	379.1

Table 3: VND results starting from the CW solution.

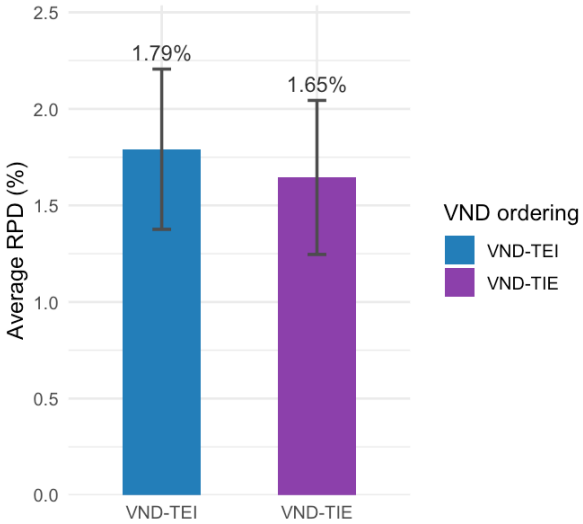

Figure 4: Average RPD (%) for VND-TEI and VND-TIE. VND-TIE is lower and more consistent. The difference is statistically significant (Wilcoxon, $p = 0.0074$).

Both VND variants beat every single-neighbourhood algorithm, though the margin is thin. Insert-First (CW) achieves 1.59% RPD; VND-TIE gets 1.64%. Insert already captures most of what local search can offer here, so VND has little room to improve on it.

VND-TIE beats VND-TEI on both quality (1.64% vs 1.79%) and speed (3.24 s vs 4.86 s), and the difference is significant ($p = 0.0074$). The ordering matters: in TIE, insert runs before exchange, so the strongest neighbourhood gets to work on a solution that exchange has not already distorted. In TEI, exchange goes first and can trap the solution in a local optimum that insert struggles to escape. Which neighbourhood you run first is not a detail.

5.5. Execution Time and Complexity

5.5.1. Why do some algorithms take much longer than others?

Execution times vary considerably in Table 1 They range from a few milliseconds to several seconds for the FI. Figure 5 visualises this spread.

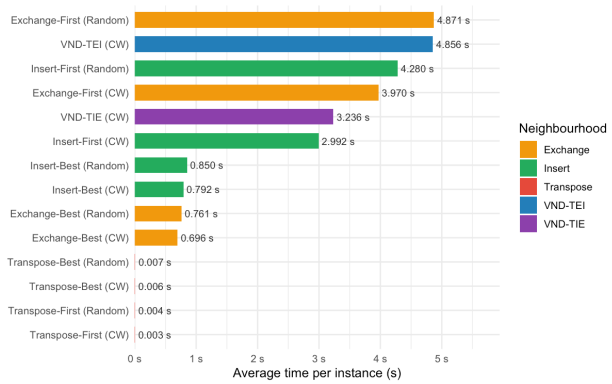

Figure 5: Average execution time per instance (s) for all configurations, sorted from fastest to slowest.

Comparing Figure 5 with Figure 1 and Table 1, the ordering is inverted, the configurations that take the longest (Insert-First, Exchange-First) are the ones with the best RPDs, and the fastest (Transpose) give the worst solutions. The reason is that a bigger neighbourhood means more candidate moves to evaluate at each step,

which costs more time but also lets the algorithm see further into the solution space. It finds better local optima because it actually looks at more alternatives before deciding. Transpose is the opposite extreme, only $n-1$ adjacent swaps per iteration, so each step is almost free, but the search barely moves and gets stuck early.

Per-iteration complexity explains the gap more precisely. Each iteration scans the neighbourhood for an improving move. The cost depends on how many moves there are and how expensive each evaluation is

Neighbourhood	Size	Cost per Δ	Cost per iteration
Transpose	$n-1$	$O(1)$	$O(n)$
Exchange	$O\left(\frac{n^2}{2}\right)$	$O(n)$	$O(n^3)$ for BI, $O(n^2)$ avg for FI
Insert	$O(n^2)$	$O(n)$	$O(n^3)$ for BI, $O(n^2)$ avg for FI

Table 4: Theoretical cost per iteration of local search for each neighbourhood.

For transpose, there are only $n-1$ pairs to check and each one costs $O(1)$, so a full scan is $O(n)$. That is why transpose runs in under 10 ms even on instances with $n = 250$: there is just not much work to do per step.

Exchange and insert have around n^2 possible moves, and evaluating each one requires scanning the elements between the two positions, which is $O(n)$. One full iteration then costs about $O(n^3)$ under best-improvement. First-improvement does not always go through the whole list since it stops as soon as it finds a positive move, but close to a local optimum almost nothing improves, so it ends up checking most of the neighbourhood anyway.

Figure 6 shows the quality/time ratio. Figure 6 doesn't contain Transpose because as for Figure 1 this would be not reachable but a version with Transpose is available in Section 7. As seen before, this version show that only y axis is grown.

Figure 6: Quality–time trade-off for all configurations (Insert, Exchange and VND).

5.5.2. Effect of initialisation on time

CW also speeds up convergence. Starting closer to a local optimum means fewer iterations to get there. For Insert-FI, the CW start cuts about 1.3 s off the runtime compared to a random start.

5.5.3. Possible improvements

One way to speed up local search without losing much quality is to use a different neighbourhood. The Chanas-Kobylanski (CK) heuristic that uses two functions sort and reverse. Each cycle is guaranteed not to worsen the solution, and CK runs faster than insert on the same instances for comparable solution quality [6].

Another direction is to escape local optima entirely. Iterated Local Search (ILS) does this by perturbing a local optimum with a few random exchanges and then re-optimising from the perturbed solution. On the LOP, ILS with CK as the inner local search finds global optima on standard benchmarks in under a few seconds [6].

6. Conclusion

All three design choices neighbourhood, initialisation, pivoting rule, affect solution quality, but not by the same amount. The neighbourhood is by far the most important: insert dominates exchange, and both dominates Transpose. CW initialisation helps everywhere, with the biggest difference for transpose where the starting point matters most since the search itself is so weak. The pivoting rule has a smaller but consistent effect: first-improvement finds better solutions at the cost of more time, while best-improvement trades a bit of quality for speed.

VND improves on the best single-neighbourhood algorithm, but VND-TIE is a bit better than Insert-First (CW). Insert already captures most of what local search can offer on this problem. VND adds something, but not much. The neighbourhood ordering does matter though: TIE beats TEI in both quality and speed, and the difference is statistically significant. Running insert before exchange is clearly better than the other way around.

References

[1] Wikipedia contributors, "Standard deviation." [Online]. Available: https://en.wikipedia.org/wiki/Standard_deviation

[2] Wikipedia contributors, "Wilcoxon signed-rank test." [Online]. Available: https://en.wikipedia.org/wiki/Wilcoxon_signed-rank_test

[3] Wikipedia contributors, "P-value." [Online]. Available: https://en.wikipedia.org/wiki/P-value

[4] R Core Team, "wilcox.test: Wilcoxon Rank Sum and Signed Rank Tests." [Online]. Available: https://stat.ethz.ch/R-manual/R-devel/library/stats/html/wilcox.test.html

[5] T. Stützle, "Slides for INFO-H-413 Heuristic Optimization." 2026.

[6] T. Schiavinotto and T. Stützle, "Search space analysis of the linear ordering problem," in *Workshops on Applications of Evolutionary Computation*, 2003, pp. 322–333.

Appendix

7. Additional Graphs

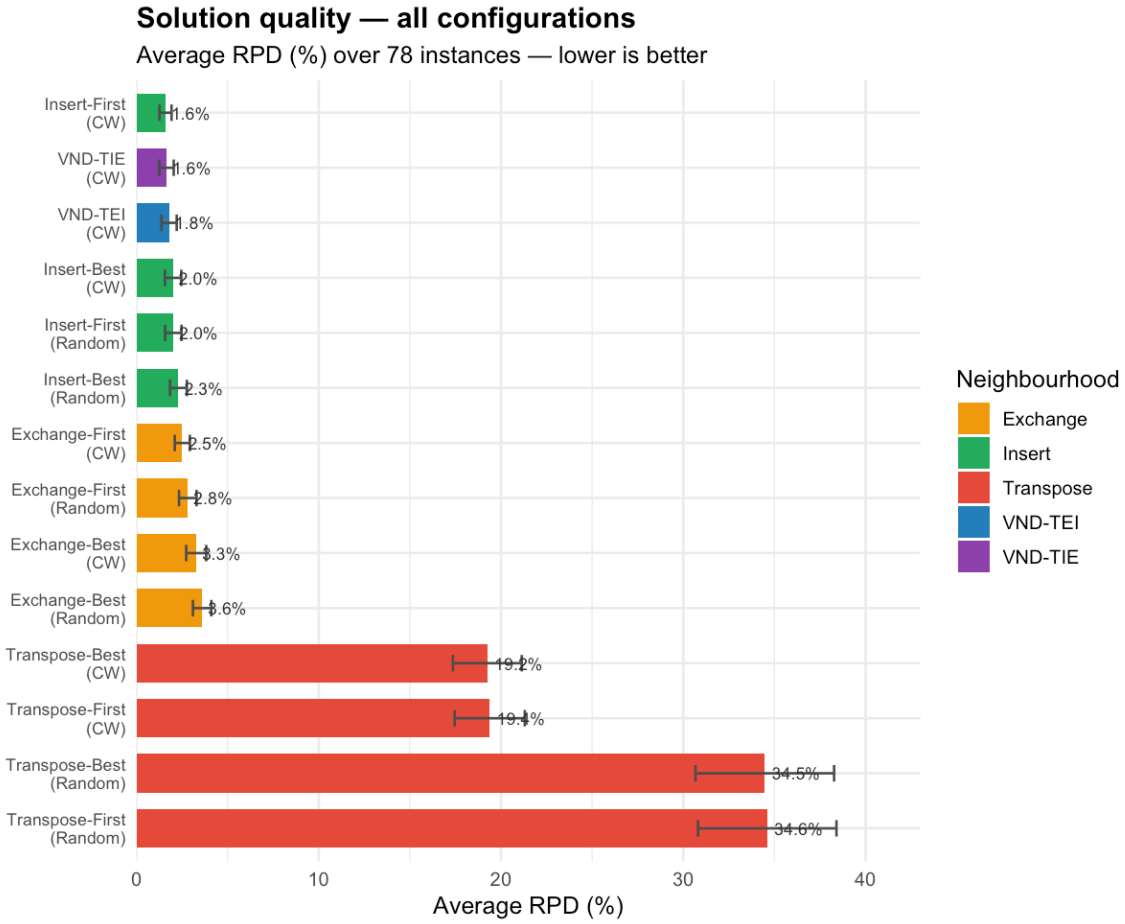

Figure 7: Average RPD (%) for Insert, Exchange and VND configurations, sorted from best to worst. Each bar shows the average over 78 instances.

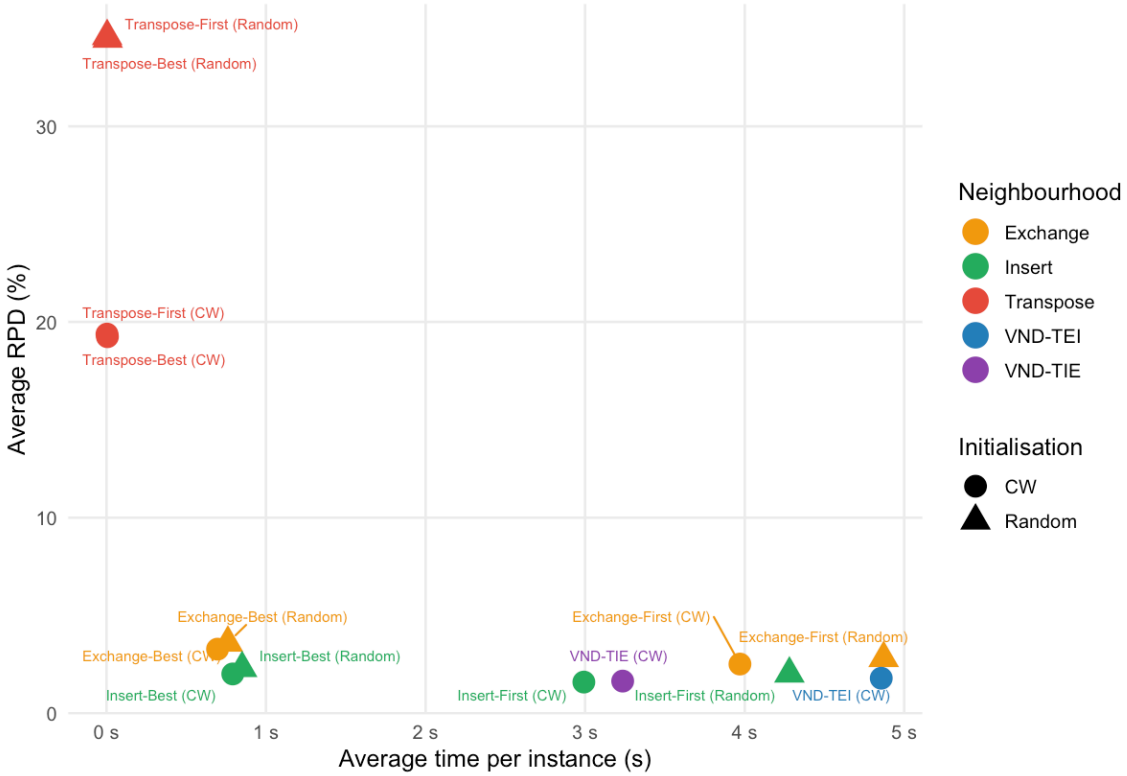

Figure 8: Quality-time trade-off for all 14 configurations including Transpose.